

Exercício Prático: API de Empréstimos da Biblioteca Lions

Turma: LionsDev

Tópicos: APIs REST com Express.js, Integração com MongoDB (Mongoose), Schemas e Modelos, Validações básicas com Mongoose, Operações CRUD assíncronas (async/await), Query Params (filtros), Regras de Negócio simples e Status Codes HTTP.

1. Contexto

A **Biblioteca Lions** controla seus empréstimos em uma planilha simples. O problema é que os registros ficam desorganizados e a equipe não consegue saber rapidamente quais materiais estão emprestados, devolvidos ou atrasados.

Você deverá criar uma API REST simples conectada ao MongoDB para registrar empréstimos de materiais da biblioteca. A estrutura deve seguir o modelo usado em aula: `server.js`, `db.js` e `models/`, com as rotas escritas diretamente no `server.js`.

2. Configuração Inicial

Crie uma API Node.js com Express e Mongoose.

Requisitos estruturais:

- Crie um arquivo `.env` contendo as variáveis `MONGO_URI` e `PORT` (porta `3000`).
- Crie a pasta `src`.
- Crie o arquivo `src/db.js` para gerenciar a conexão com o MongoDB.
- Crie o arquivo `src/server.js` como ponto de entrada da aplicação.
- Crie a pasta `src/models` e nela o arquivo `emprestimo.js`.
- Todas as rotas devem ficar no `src/server.js`, como fizemos em sala.

O Modelo (Schema) do Empréstimo

O Schema do Mongoose para os `emprestimos` deve conter os seguintes campos:

- **tituloMaterial**: Tipo **String**, obrigatório.
- **tipoMaterial**: Tipo **String**, obrigatório (deve aceitar apenas: **Livro**, **Revista** ou **Apostila**).
- **nomeAluno**: Tipo **String**, obrigatório.
- **turma**: Tipo **String**, obrigatório.
- **dataEmprestimo**: Tipo **String** ou **Date**, obrigatório (ex: **"2026-06-15"**).
- **diasEmprestimo**: Tipo **Number**, obrigatório.
- **dataDevolucaoPrevista**: Tipo **String** (será calculado automaticamente pela API).
- **multaPrevista**: Tipo **Number** (será calculado automaticamente pela API).
- **status**: Tipo **String**, com valor padrão de **"Emprestado"** (deve aceitar apenas: **Emprestado**, **Devolvido** ou **Atrasado**).

3. Requisitos Obrigatórios e Regras de Negócio

3.1 Cadastrar Empréstimo (CREATE)

Crie a rota **POST /emprestimos**.

O corpo da requisição enviará os dados do empréstimo (exceto **dataDevolucaoPrevista**, **multaPrevista** e **status**).

```
{
  "tituloMaterial": "Lógica de Programação",
  "tipoMaterial": "Livro",
  "nomeAluno": "Mariana Souza",
  "turma": "LionsDev",
  "dataEmprestimo": "2026-06-15",
  "diasEmprestimo": 10
}
```

Regras:

1. **Cálculo da Data de Devolução**: Some **diasEmprestimo** à **dataEmprestimo** e salve o resultado em **dataDevolucaoPrevista**.
2. **Multa Prevista**: A biblioteca permite até 7 dias de empréstimo sem multa. Se **diasEmprestimo** for maior que 7, cobre R\$ 2 por dia extra.
3. **Retorno**: Salve o empréstimo no banco e retorne o documento criado com status **201**.

3.2 Listar Todos os Empréstimos (READ)

Crie a rota `GET /emprestimos`.

Regras:

- A rota deve retornar todos os empréstimos salvos no MongoDB usando `Emprestimo.find()`.
- Retorne o array de resultados com status `200`.

3.3 Buscar Empréstimos por Aluno (QUERY PARAMS)

Crie a rota `GET /emprestimos/busca`.

Esta rota deve aceitar um filtro opcional via Query Params chamado `aluno`:

- Exemplo de URL: `http://localhost:3000/emprestimos/busca?aluno=mari`
- A busca deve retornar todos os empréstimos em que o nome do aluno contenha o texto pesquisado.

3.4 Atualizar Status do Empréstimo (UPDATE)

Crie a rota `PATCH /emprestimos/:id`.

O corpo da requisição deve enviar apenas o novo status (ex: `{ "status": "Devolvido" }`).

Regras:

- Busque o empréstimo pelo ID e atualize o status usando `findByIdAndUpdate`.
- Se o ID não for encontrado, responda com status `404`.
- Retorne o documento atualizado com status `200`.

3.5 Remover Empréstimo (DELETE)

Crie a rota `DELETE /emprestimos/:id`.

Regras:

- Remova o empréstimo do banco de dados pelo ID usando `findByIdAndDelete`.
- Se não encontrar o registro, retorne status `404`.
- Se remover com sucesso, retorne status `200` com uma mensagem de confirmação.

4. Testes Esperados

Realize testes na sua API seguindo este fluxo de validação:

1. Cadastre um empréstimo de 5 dias e verifique se a `multaPrevista` ficou R\$ 0.
 2. Cadastre um empréstimo de 10 dias e verifique se a `multaPrevista` ficou R\$ 6.
 3. Liste todos os empréstimos cadastrados.
 4. Faça uma busca pelo nome de um aluno.
 5. Atualize o status de um empréstimo para `Devolvido`.
 6. Delete um dos empréstimos informando o ID.
-

5. Dicas para a Implementação

- Para somar dias a uma data, crie um `new Date(dataEmprestimo)` e use `setDate`.
 - Para calcular a multa, use uma estrutura `if`.
 - No `findByIdAndUpdate`, passe `{ new: true, runValidators: true }`.
 - Lembre-se de usar `app.use(express.json())` no `src/server.js` para conseguir ler o corpo das requisições.
-

LionsDev • Professor Nicolas Cardoso Motta
Exercício Prático de Mongoose e MongoDB - Módulo 08